Generative Project

Text-To-Image Generation Using Transformer And Diffusion Models

Milestone 2

Group 02

Abdellah Faleh
Jin-Woo Hong
Samuel Tong
Tristan Lyons

faleh.a@northeastern.edu
hong.jinw@northeastern.edu
tong.sam@northeastern.edu
lyons.tr@northeastern.edu

Git Repo Link: https://github.com/abdul6253/IE7615-group-02-project2

Submission Date: 11/15/2026

1. Project Overview and Objectives

Project 2 builds a system that makes a new picture from a short caption. It has two parts: a **text encoder** (a Transformer such as CLIP or BERT) that turns the caption into numbers, and a **diffusion model** (Stable Diffusion 1.5) that turns random noise into a picture, guided by those numbers. In Milestone 1 we prepared 2,000 Flickr30k photo–caption pairs, checked CLIP, and made first images with the ready-made pipeline.

Goals for Milestone 2:

- Connect the text encoder to the diffusion model **ourselves**, and show that the text really controls the image.
- Run conditional generation on a fixed set of test prompts, saving every image with its prompt and seed.
- Test the **guidance scale** (6 values) and the **sampler, number of steps and noise schedule**.
- Train parts of the pipeline (a BERT adapter and LoRA for the U-Net) and record losses, time, GPU memory and problems.

We did not rebuild anything from Milestone 1: both new notebooks load the same 2,000 captions (check code 477c680542) and the same 15 test prompts.

2. Methodology

2.1 Our choice: a pretrained pipeline with our own conditioning path

The assignment allows two tracks: train a small diffusion model from scratch, or build on a pretrained conditional pipeline and change its conditioning path. We chose the second. A from-scratch model on a free T4 GPU would only reach 32×32 or 64×64 pixels and would need many hours of training, while our goal is to study how text controls good 512×512 images. So we kept the trained U-Net and VAE **frozen** and changed the parts around them: (1) we replaced the ready-made pipeline with **our own denoising loop** that takes text embeddings as input, and (2) we trained a **BERT → CLIP adapter** so BERT can be used as the text encoder instead of CLIP, and (3) we fine-tuned the U-Net itself with small **LoRA** adapters on our training photos (Section 2.4).

2.2 How the text reaches the image

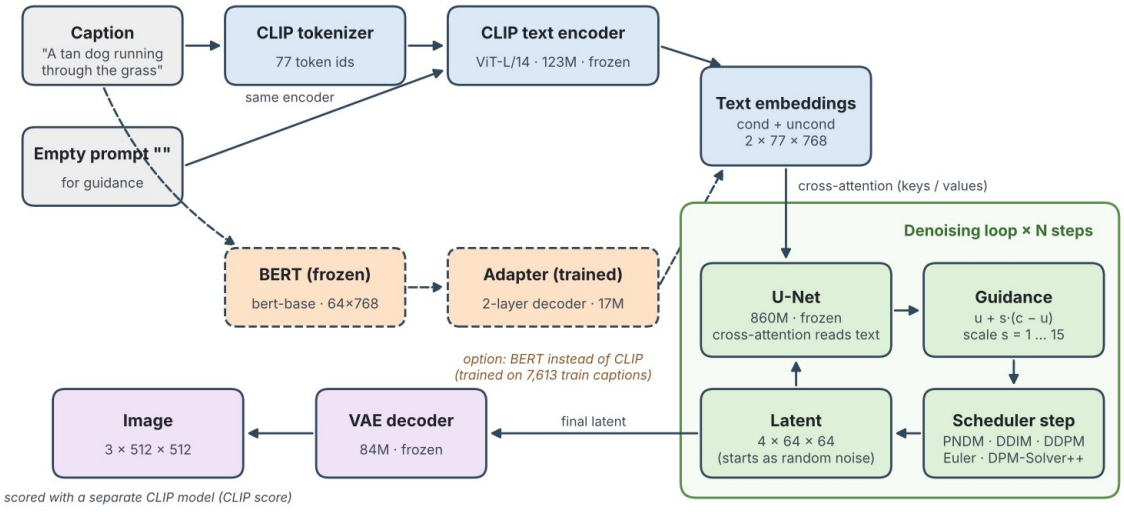


Figure 1. Our integration: text → embedding → cross-attention in the U-Net → latent → image. Dashed boxes: the BERT path with our trained adapter.

Figure 1 shows the full path. The CLIP tokenizer cuts the caption into 77 tokens, and the CLIP text encoder turns them into a **77×768 embedding**, one vector per token. The empty prompt "" is encoded too. Both go into the U-Net together (2×77×768). Inside the U-Net, **cross-attention** layers use the text vectors as keys and values, so every part of the image can "look at" the words. The U-Net's text size (768) must match the encoder's size, and we check this in code.

The image starts as random noise in a small **latent** of 4×64×64 numbers, 48 times smaller than the final image. At each step the U-Net predicts the noise twice, with the empty prompt (u) and with our caption (c). **Classifier-free guidance** [4] mixes them: $noise = u + s \cdot (c - u)$, where s is the guidance scale. The **scheduler** then removes that noise. After N steps

the VAE decoder turns the latent into a 512×512 picture.

2.3 The BERT → CLIP adapter (the part we trained)

BERT [10] also gives 768 numbers per token, so it "fits" the U-Net, but the U-Net was trained only with CLIP, so it cannot read BERT's numbers. Our adapter translates them. It has 77 learned query vectors and a 2-layer Transformer decoder that reads the BERT tokens through cross-attention, then outputs a 77×768 embedding (17M parameters). Only the adapter learns; BERT, CLIP, the U-Net and the VAE stay frozen.

Training setup: 7,613 cleaned captions of the 1,600 train photos (up to 5 per photo); 200 val captions for validation (test captions never used); target = CLIP's embedding of the same caption, mean squared error on standardized values; 15 epochs, batch 64, AdamW, learning rate 3×10^{-4} (one-cycle), seed 42.

2.4 LoRA fine-tuning of the U-Net

To adapt the image model itself to Flickr30k, we added **LoRA** adapters [12] (rank 8) to the U-Net's attention layers (query, key, value and output). Only these small matrices learn: **1.6M of the U-Net's 861M parameters**. Each training step takes one of the 1,600 **train** photos and its caption, turns the photo into a latent with the frozen VAE, adds noise at a random step, and teaches the U-Net to predict that noise while reading the caption through cross-attention (mean squared error). Settings: 800 steps, batch size 1, learning rate 10^{-4}, seed 42 (notebook *04_sd_lora_finetuning*, scripts *train_sd_lora.py* and *evaluate_sd_conditioning.py*).

2.5 Experiments

All images are 512×512 and use seed *42 + prompt id* unless noted. For every image we save the prompt, seed and settings, and record the **CLIP score** (0–100, how well the image matches the caption [2]), the time, the GPU memory and the **color saturation**. Four sweep prompts were picked to mix easy and hard cases: a dog (#0), a soccer player in a "number ten" jersey (#2), "five people on bikes" (#6) and a rave with glow sticks (#10).

Part	What changes	Images
A. Baseline	nothing: 15 test prompts × 2 seeds; PNDM, 30 steps, guidance 7.5	30
B. Guidance scale	1, 3, 5, 7.5, 10, 15 (4 prompts × 3 seeds)	72
C. Sampler × steps	DDPM [5], DDIM [6], PNDM [7], Euler [9], DPM-Solver++ [8] × 10, 20, 30, 50 steps	80
D. Noise schedule	scaled linear (trained), linear, cosine [11] (DDIM, 30 steps); default vs. Karras [9] sigmas (DPM-Solver++, 20 steps)	20

Table 1. Generation experiments (202 images, notebook 05).

3. Implementation and Results

The work is in three new notebooks: *04_integration_training* (our loop, checks and adapter training; 10.3 minutes on a T4), *05_conditional_generation_sweeps* (all generation experiments; 20.9 minutes) and *04_sd_lora_finetuning* (LoRA). The first two save their logs, images and a *summary.json* in the repository.

3.1 The integration works

- **Same result as the official pipeline:** for the same prompt and seed, our loop and the official pipeline give the same image (mean pixel difference **0.0** out of 255). It is also identical to the Milestone 1 image.
- **The text controls the image:** we made 6 images with the **same seed** but different captions, so only the text changed (Figure 2). CLIP matched **6 of 6** images to their own caption. The average score was 27.4 with the right caption and 7.8 with a wrong one. The empty prompt gave a random sketch-like pattern (11.3).



Figure 2. Same seed (42) for every image: only the caption changes. Last image: empty prompt.

3.2 Training the adapter

Training took **5.1 minutes** (20.5 s per epoch, 1,785 steps) and used at most **6.35 GB** of GPU memory. The validation loss fell from 0.85 to **0.52** (best epoch 12). A model that always outputs the average embedding scores 1.00, so the adapter really learned. The closeness to CLIP's embeddings (cosine similarity) rose from 0.61 to 0.78. After about epoch 10 the training loss kept falling (0.34) but the validation loss stayed flat: mild overfitting.

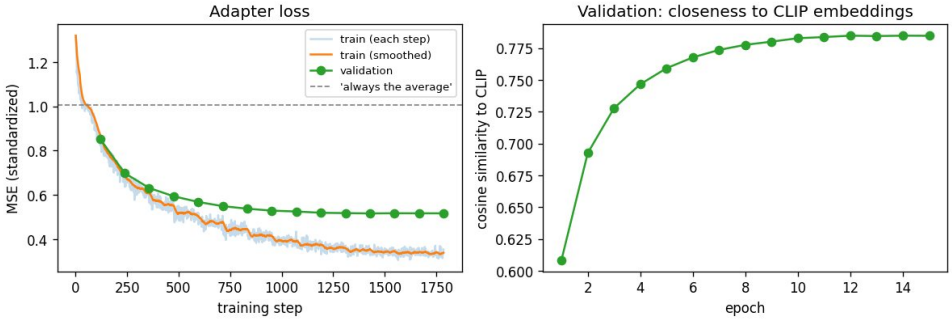


Figure 3. Adapter loss (left) and closeness to CLIP on the validation captions (right).

3.3 CLIP vs. BERT as the text encoder

Mean CLIP score on 6 test prompts (same seeds): **CLIP 28.8**, **raw BERT 9.9** (random patterns, rooms and collages: the text is ignored) and **BERT + our adapter 21.4** (4 of 6 clearly right; the rave and the roller coaster are wrong).

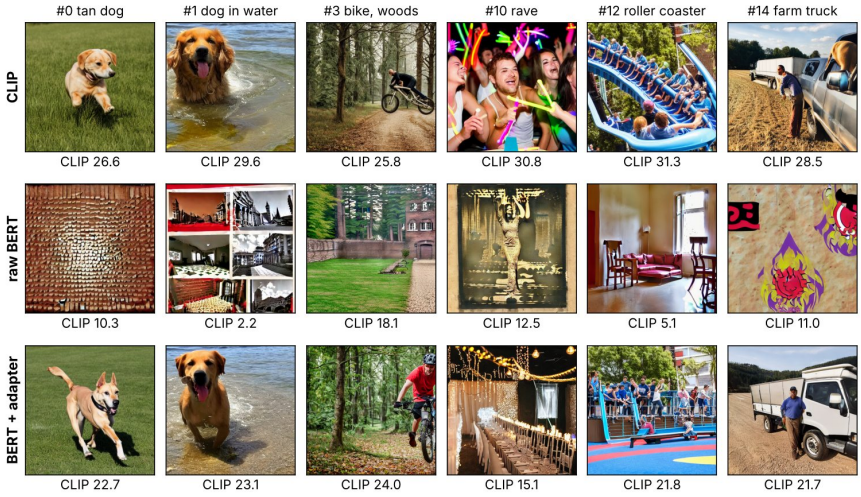


Figure 4. CLIP (top), raw BERT (middle) and BERT + adapter (bottom) for 6 test prompts.

3.4 LoRA fine-tuning

The LoRA run finished all 800 steps on the T4. The loss (average of every 25 steps) stayed between 0.10 and 0.24, 0.19 for the first 200 steps and 0.19 for the last 200, so it did **not clearly go down**. This is common for diffusion training with batch size 1, because each step uses a different random noise level, and 800 steps is a short first run. The preview for a validation caption ("Three boys wearing green shirts and tan pants pose at the top of a slide") shows a clear, realistic slide scene, but with two boys, white shirts and green shorts: the same counting and attribute mistakes as the base model. The held-out comparison (pretrained vs. LoRA vs. empty prompt) is written but not yet run; it moves to Milestone 3.

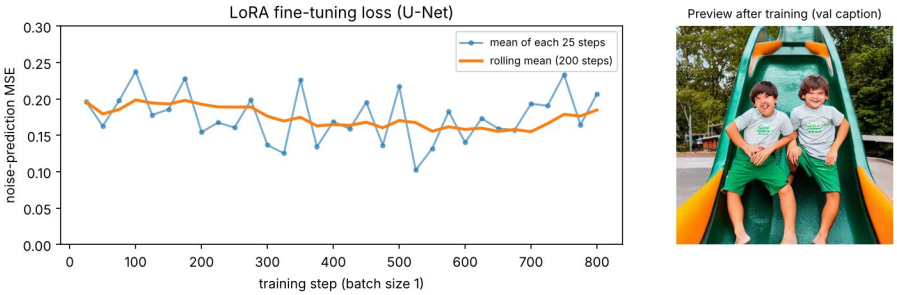


Figure 5. LoRA loss during 800 steps (left) and the preview image after training (right).

3.5 Baseline conditional generation

The 30 baseline images have a mean CLIP score of **27.3** and took **5.0 s** each (peak GPU memory 4.3 GB). The safety filter blocked none of the 202 images. The seed-42 images are identical to Milestone 1, so passing our own embeddings gives exactly the same result as passing text.

3.6 Guidance scale

Guidance scale	1	3	5	7.5	10	15
Mean CLIP score	20.8	24.8	25.8	**26.0**	25.6	25.9
Similarity between seeds (higher = less variety)	68.2	81.3	84.8	86.6	87.3	89.6
Color saturation	30.7	34.2	39.9	46.4	50.8	57.5
Seconds per image	3.4	5.7	5.7	5.6	5.6	5.7

Table 2. Guidance sweep (4 prompts × 3 seeds per scale).



Figure 6. One seed per prompt at each guidance scale (PNDM, 30 steps).

3.7 Sampler and number of steps

Time grows in a straight line with the number of steps, about **0.19 s per step** for every sampler (the U-Net does almost all the work), from 2.0 s at 10 steps to 9.0 s at 50. Memory was 4.3 GB for all. The CLIP score hardly changed (25.5 to 27.4), even for clearly broken images, so we also measured how far each image is from the same sampler's 50-step image (mean pixel difference; lower = the image has settled).

Sampler	10 steps	20 steps	30 steps	Mean CLIP (all steps)
DPM-Solver++	17.2	**9.2**	4.8	26.2
Euler	18.7	10.2	6.6	26.5
DDIM	24.6	13.8	11.8	26.2
PNDM (default)	31.3	16.0	14.6	26.5
DDPM	44.4	38.8	35.4	26.1

Table 3. Difference from the same sampler's 50-step image (0–255; 4 prompts).

3.8 Noise schedule

Schedule	Mean CLIP	Saturation	What we see
scaled linear (trained), DDIM 30	25.8	41.7	reference
linear, DDIM 30	26.0	54.4	same scenes, stronger and greener colors
cosine, DDIM 30	25.3	45.1	blurry and washed out; faces lose detail
DPM-Solver++ 20, default sigmas	25.9	45.0	sharp, 3.7 s per image
DPM-Solver++ 20, Karras sigmas	25.6	43.9	almost the same as default

Table 4. Noise schedule test (4 prompts, seed 42 + id).

4. Analysis

4.1 What worked

- **The conditioning path works and is exact:** our own loop reproduces the pipeline to the pixel, and the same noise turns into six different, correct scenes when only the caption changes.
- **Guidance 5 to 7.5 is the best range.** Scale 1 ignores much of the caption (the dog hides in tall grass, the rave becomes a collage) but gives the most variety. From 3 to 7.5 the match improves quickly. Above 7.5 the match does not improve, while colors get stronger (saturation 46 → 58) and the seeds look more alike (86.6 → 89.6). This is the trade-off between prompt match and diversity that classifier-free guidance is known for [4].
- **Faster sampling is possible:** DPM-Solver++ and Euler have settled by 20 steps, so DPM-Solver++ at 20 steps (3.8 s) gives the same look as our PNDM baseline at 30 steps (5.7 s), about **one third faster**.
- **LoRA training runs on a free GPU:** adapting only 0.2% of the U-Net's weights fits on a T4, so we can now fine-tune the image model on our own data.
- **The adapter makes BERT usable:** it closes about 60% of the gap between raw BERT (9.9) and CLIP (28.8), after only 5 minutes of training on 7,613 captions.

4.2 What did not work

- **Few steps break some samplers:** PNDM at 10 steps gives a colorful, broken dog, and DDIM at 10 steps is blurry.
- **DDPM never settles:** it adds new random noise at every step, so each step count gives a *different* picture (difference 35–44). It is also the slowest to reach a stable result.
- **The noise schedule must match training:** the cosine schedule makes blurry, washed-out images, because the U-Net learned with the scaled linear schedule. Linear changes the colors. Karras sigmas changed almost nothing at 20 steps.
- **Guidance does not fix Milestone 1 mistakes:** the "number ten" jersey shows "111" at every scale, and "five people on bikes" shows three or four riders and no man watching at every scale.
- **LoRA showed no clear gain yet:** the loss stayed flat over 800 steps and the preview repeats the base model's counting and color mistakes; we cannot say it is better until the held-out comparison is run.
- **The adapter loses rare details:** "glow sticks" became a dining hall with lights and the roller coaster became a playground. With only 7,613 captions, rare words are not learned well, and the gap between training and validation loss shows overfitting.

4.3 Problems we met

- **CLIP score is not enough:** it stayed near 26 for broken images (PNDM at 10 steps scored 27.8) and even favored a less detailed DDPM image (33.5). We added the difference-to-50-steps measure and checked every grid by eye. Milestone 3 will add FID.
- **A code bug:** the adapter crashed once when given BERT's 0/1 padding mask instead of a True/False mask; we fixed it with one change and reran only that cell.

4.4 Ethics

All images come from a model trained on internet data and can repeat stereotypes (for example, who appears at a party or on a farm). We kept the safety filter on (0 of 202 images blocked), used only test captions for the galleries, and use the images only for this course.

4.5 Next steps (Milestone 3)

- Compute FID and Inception Score for the baseline and the best settings, and run the parameter-sensitivity study over guidance scale, sampler and steps, noise schedule and text encoder (CLIP vs. BERT + adapter).
- Use DPM-Solver++ with 20 steps for the large FID runs, since it saves about a third of the time.
- Run the LoRA held-out comparison, train longer (and with a larger batch) and compare it with the base model.
- Improve the adapter with early stopping and more data (more Flickr30k captions), then measure it again.

5. Team Contributions

Member	Role	Contribution and responsibility
Abdellah Faleh	Technical lead	Built and ran notebooks 04_integration_training and 05 (integration, BERT adapter, all sweeps) and wrote the integration write-up, samples gallery, training summary and this report
Jin-Woo Hong	Experiments	Built and ran the LoRA fine-tuning of the U-Net (notebook 04_sd_lora_finetuning, training and evaluation scripts) and updated notebook 03 to pass explicit CLIP embeddings; leads the parameter-sensitivity study in Milestone 3
Samuel Tong	Evaluation	Reviews the CLIP-score results and the adapter comparison; leads FID, Inception Score and alignment analysis in Milestone 3
Tristan Lyons	Documentation	Keeps the GitHub repository organized, submits on Canvas, and is the main contact with the teaching staff

References

[1] P. Young, A. Lai, M. Hodosh and J. Hockenmaier, "From image descriptions to visual denotations," *TACL*, vol. 2, pp. 67–78, 2014.

[2] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021.

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. CVPR*, 2022.

[4] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *NeurIPS Workshop on Deep Generative Models*, 2021.

[5] J. Ho, A. Jain and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NeurIPS*, 2020.

[6] J. Song, C. Meng and S. Ermon, "Denoising diffusion implicit models," in *Proc. ICLR*, 2021.

[7] L. Liu, Y. Ren, Z. Lin and Z. Zhao, "Pseudo numerical methods for diffusion models on manifolds," in *Proc. ICLR*, 2022.

[8] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, "DPM-Solver++: Fast solver for guided sampling of diffusion probabilistic models," arXiv:2211.01095, 2022.

[9] T. Karras, M. Aittala, T. Aila and S. Laine, "Elucidating the design space of diffusion-based generative models," in *Proc. NeurIPS*, 2022.

[10] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in *Proc. NAACL*, 2019.

[11] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. ICML*, 2021.

[12] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.